PyTorch & Réseaux de Neurones Convolutifs

Jour 3 — Après-midi

Julien Rolland

Formation M2 Développement Fullstack

Jour 3

Plan du module

1 PyTorch

2 CNN

Pourquoi passer à PyTorch ?

Limite du « fait main » (J3-AM)

- Notre classe `Value` est **pédagogique** mais lente
- Python pur : quelques milliers d'opérations/s
- Un vrai réseau : **milliards** d'opérations par forward

Même principe, autre échelle

Même graphe de calcul, même `backward()`
$\Rightarrow$ mais sur des **tenseurs** en C++/CUDA

La solution industrielle

- **Calcul vectorisé** : matrices entières en une opération
- **Backend C++/CUDA** : parallélisme GPU
- **Écosystème riche** :
  - `torchvision` — Vision par ordinateur
  - `torchaudio` — Audio
  - `transformers` — NLP / LLM

Le Concept de Tenseur (torch.Tensor)

$$\texttt{torch.Tensor} = \texttt{np.ndarray} + \textbf{autograd}$$

np.ndarray

```python
import numpy as np

x = np.array([[1.0, 2.0],
              [3.0, 4.0]])
y = x ** 2
print(y.sum())  # 30.0
# Pas de gradient !
```

torch.Tensor

```python
import torch

x = torch.tensor([[1.0, 2.0],
                  [3.0, 4.0]],
                 requires_grad=True)
y = x ** 2
y.sum().backward()
print(x.grad)  # 2*x
```

NumPy ↔ PyTorch : Même API

np.ndarray

```python
x.shape          # (3, 4)
x.dtype          # float64

np.ones((3, 4))
np.zeros((3, 4))
np.rand(3, 4)

x + y
x @ y            # produit matriciel
x.T              # transposee

x.reshape(2, 6)
x[0, :]          # slicing
```

torch.Tensor

```python
x.shape          # torch.Size([3, 4])
x.dtype          # torch.float32

torch.ones(3, 4)
torch.zeros(3, 4)
torch.rand(3, 4)

x + y
x @ y            # produit matriciel
x.T              # transposee

x.reshape(2, 6)
x[0, :]          # slicing
```

Nouveaux concepts : requires_grad et device

requires_grad : suivi du calcul

- Marque un tenseur comme **variable à optimiser**
- PyTorch enregistre toutes les opérations le concernant
- `.backward()` remonte le graphe et calcule `.grad`

```python
x = torch.tensor([2.0], requires_grad=True)
y = x ** 3
y.backward()
print(x.grad)  # 3*x^2 = 12.0
```

device : CPU ou GPU

- Les tenseurs vivent sur un **device**
- Les opérations exigent le **même device**
- `.to(device)` déplace un tenseur ou un modèle

```python
device = (
    "cuda" if torch.cuda.is_available()
    else "cpu"
)
x = x.to(device)
model = model.to(device)
```

La Philosophie `torch.nn`

Le `Module` : brique de base

- On ne gère **plus W** et ***b*** manuellement
- `nn.Linear`, `nn.Conv2d`... gèrent leurs paramètres
- `__init__` : définit les couches
- `forward()` : flux de données

Avantage

`model.parameters()` retourne **tous** les poids ⇒ prêts pour l'optimiseur

```python
import torch.nn as nn
import torch.nn.functional as F

class MLP(nn.Module):
    def __init__(self, n_in, n_h, n_out):
        super().__init__()
        self.fc1 = nn.Linear(n_in, n_h)
        self.fc2 = nn.Linear(n_h, n_out)

    def forward(self, x):
        x = F.relu(self.fc1(x))
        return self.fc2(x)

model = MLP(784, 128, 10)
# Nombre de parametres :
n = sum(p.numel() for p in model.parameters())
print(f"{n} parametres")  # 101770
```

Optimisation : `torch.optim`

Plus de `w -= lr*grad` manuel

- **SGD** : descente de gradient classique
- **Adam** : adaptatif, converge rapidement
- **RMSprop** : stable pour les RNN

Interface unifiée

Peu importe l'optimiseur, l'API est toujours la même : `opt.zero_grad()` / `opt.step()`

```python
import torch.optim as optim

# SGD classique
opt = optim.SGD(model.parameters(), lr=1e-2, momentum=0.9)

# Adam (defaut recommande)
opt = optim.Adam(model.parameters(), lr=1e-3)

# Meme interface pour tous :
opt.zero_grad()
loss.backward()
opt.step()
```

Data : Dataset & DataLoader

Dataset : accès aux données

- Définit **comment** accéder à un exemple
- Implémente `__len__` et `__getitem__`
- `torchvision.datasets` fournit MNIST, CIFAR...

DataLoader : pipeline

- Gère le **batching** et le **shuffling**
- `num_workers` : chargement en parallèle
- Le GPU ne doit pas attendre le CPU

```python
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

dataset = datasets.MNIST(
    root="data", train=True,
    transform=transforms.ToTensor(),
)

loader = DataLoader(
    dataset,
    batch_size=32,
    shuffle=True,
    num_workers=4,
)

for X, y in loader:
    # X.shape = [32, 1, 28, 28]
    pass
```

La Training Loop Standard

Les 5 étapes rituelles

1. `zero_grad()` : efface les anciens gradients
2. `model(x)` : forward pass
3. `criterion(out, y)` : calcul de la perte
4. `loss.backward()` : autograd
5. `optimizer.step()` : mise à jour des poids

Piège classique

Oublier `zero_grad()` ⇒ gradients **accumulés** ⇒ divergence

```python
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters())

for epoch in range(n_epochs):
    for X, y in train_loader:
        # 1. Zero gradients
        optimizer.zero_grad()

        # 2. Forward pass
        output = model(X)

        # 3. Compute loss
        loss = criterion(output, y)

        # 4. Backward (autograd)
        loss.backward()

        # 5. Update weights
        optimizer.step()
```

Pourquoi le MLP Échoue sur les Images ?

Problème 1 : Explosion des paramètres

- Image $1024 \times 1024 \Rightarrow 10^6$ entrées
- Couche cachée de 512 neurones
- $10^6 \times 512 = \mathbf{5 \times 10^8}$ poids
- Rien que pour la **première couche** !

Problème 2 : Perte de topologie

- Flatten $\Rightarrow$ perte de la **structure 2D**
- Un décalage d'1 pixel change tout le vecteur
- Le réseau ne voit pas la **proximité spatiale**

Solution : CNN

- **Localité** : chaque neurone voit une petite zone
- **Partage des poids** : même filtre sur toute l'image
- **Hiérarchie** : bords $\rightarrow$ formes $\rightarrow$ objets

	MLP	**CNN**
Params (1re couche)	500M	$\sim$ 1K
Invariance translation	Non	Oui
Localité spatiale	Non	Oui

Architecture Globale d'un CNN

Partie 1 : Extraction de features

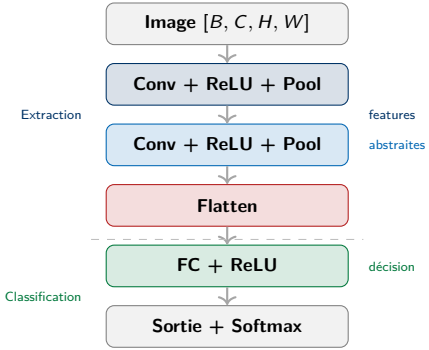
- **Conv + ReLU + Pool** $\times N$
- Image $\rightarrow$ cartes de caractéristiques abstraites
- Taille spatiale $\downarrow$, profondeur (canaux) $\uparrow$

Partie 2 : Classification

- **Flatten** : cartes $\rightarrow$ vecteur 1D
- MLP classique (J2) pour la décision finale
- Sortie : logits $\rightarrow$ softmax $\rightarrow$ classes

MNIST : $[B, 1, 28, 28] \rightarrow [B, 10]$

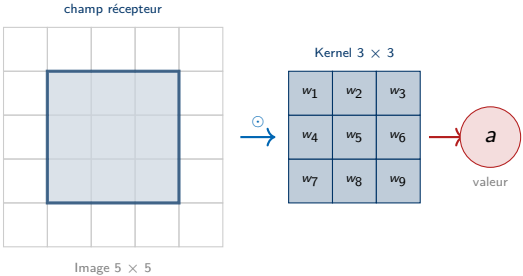
1 canal, 28×28 pixels, 10 classes



La Convolution : L'Opération de Base

Le Kernel (Filtre)

- Petite fenêtre ($K \times K$, ex: 3×3) qui **glisse** sur l'image
- **Localité** : chaque neurone ne voit qu'une zone
- **Partage des poids** : même filtre partout $\Rightarrow$ beaucoup moins de paramètres



Calcul d'une valeur de sortie

$$a = \sum_{i,j} w_{ij} \cdot x_{ij} + b$$

Produit élément par élément entre le kernel et le champ récepteur, puis somme + biais

Stride, Padding et Taille de Sortie

Stride S : pas du filtre

- $S = 1$: filtre avance pixel par pixel (défaut)
- $S = 2$: saute un pixel $\Rightarrow$ sortie $2\times$ plus petite

Padding P : zéros autour

- $P = 0$: sortie rétrécie de $K-1$ de chaque côté
- $P = \lfloor K/2 \rfloor$: **same padding**, taille préservée

Taille de sortie

$$H_{out} = \left\lfloor \frac{H_{in} - K + 2P}{S} \right\rfloor + 1$$

```python
nn.Conv2d(
    in_channels  = 1,    # canaux entree
    out_channels = 32,   # nb de filtres
    kernel_size  = 3,    # K x K
    stride       = 1,    # S
    padding      = 1,    # P -> same padding
)
# H_in=28, K=3, P=1, S=1
# H_out = (28 - 3 + 2) / 1 + 1 = 28
```

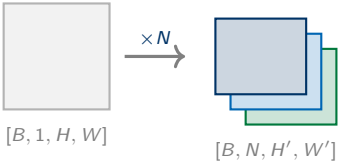
Plusieurs Filtres → Feature Maps

N filtres = N feature maps

- Chaque filtre détecte un **pattern différent**
- N filtres ⇒ tenseur de sortie $[B, N, H', W']$
- `out_channels = N` dans `nn.Conv2d`

Ce que le réseau apprend

Couche 1 : bords, contrastes
Couche 2 : coins, textures
Couche n : formes, objets...



Le Pooling : Réduction et Invariance

Max Pooling 2×2

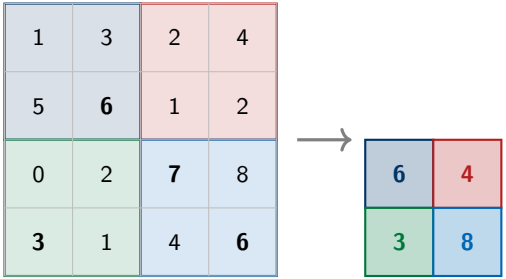
On garde la valeur **maximale** dans chaque zone 2×2

Objectifs

1. **Réduire** la résolution spatiale (moins de calculs)
2. **Invariance** aux petites translations
3. **Élargir** le champ de vision des couches suivantes

Résultat

Pool 2×2 : $H \times W \rightarrow \frac{H}{2} \times \frac{W}{2}$
$\Rightarrow 4\times$ moins de valeurs



Feature map 4×4 Après pool 2×2

CNN en PyTorch : Schéma d'Architecture

```python
class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        # --- Extraction de features ---
        self.conv1 = nn.Conv2d(..., ..., kernel_size=...)
        self.conv2 = nn.Conv2d(..., ..., kernel_size=...)
        self.pool  = nn.MaxPool2d(kernel_size=2)
        # --- Tete de classification ---
        self.fc1   = nn.Linear(..., ...)
        self.fc2   = nn.Linear(..., n_classes)

    def forward(self, x):       # x : [B, C, H, W]
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.flatten(start_dim=1)  # [B, ...]
        x = F.relu(self.fc1(x))
        return self.fc2(x)      # logits [B, n_classes]
```

Lecture du flux

1. conv1+ReLU+pool → feature maps, résolution ÷2 **2.** conv2+ReLU+pool → features abstraites, ÷2 **3.** flatten : 3D → 1D

4. fc1+ReLU : couche dense **5.** fc2 : logits $\in \mathbb{R}^{n_classes}$

Récapitulatif — Ce qu'on a vu

PyTorch

- `Tensor` = `ndarray` + autograd (`requires_grad`, `device`)
- `nn.Module` : couches, `forward()`, `parameters()`
- `torch.optim` : SGD, Adam — même interface
- `Dataset` / `DataLoader` : batching, shuffling
- Training loop : `zero_grad` / forward / loss / `backward` / `step`

CNN

- MLP : explosion des paramètres, perte de topologie
- Convolution : localité, partage des poids, kernel $K \times K$
- Stride S, padding P — formule H_{out}
- N filtres $\rightarrow$ N feature maps $[B, N, H', W']$
- Pooling : réduction $\times 2$, invariance aux translations
- Architecture : Conv+ReLU+Pool $\rightarrow$ Flatten $\rightarrow$ FC